

PROJECT REPORT

CVM++

Custom Compiler & Stack-Based Virtual Machine

A complete language runtime built from scratch in C++17

Student	Nimit Jain
Institute	IIT Guwahati
Programme	Even Semester Projects 2026
Language	C++17
GitHub	github.com/Nimit2504/CVM-plus-plus

Contents

1	Abstract	3
2	Problem Statement	3
3	System Architecture	3
3.1	Pipeline Overview	3
3.2	Data Flow Diagram	3
4	Component Design	4
4.1	Lexer — <code>lexer.h</code> / <code>lexer.cpp</code>	4
4.2	Parser — <code>parser.h</code> / <code>parser.cpp</code>	5
4.3	Abstract Syntax Tree — <code>ast.h</code>	5
4.4	Bytecode Compiler — <code>compiler.h</code> / <code>compiler.cpp</code>	5
4.4.1	Jump Backpatching	6
4.4.2	Bytecode Encoding	6
4.4.3	Compiled If Statement — Bytecode Layout	6
4.4.4	Compiled While Statement — Bytecode Layout	6
4.5	Instruction Set Architecture (ISA)	7
4.6	Virtual Machine — <code>vm.h</code> / <code>vm.cpp</code>	7
4.6.1	Value Type	7
4.6.2	Error Handling	7
5	Language Specification	8
5.1	Data Types	8
5.2	Operators	8
5.3	Full Grammar	8
5.4	Language Example	8
6	Test Results	9
6.1	Sample Program: Fibonacci Sequence	10
6.2	Debug Mode Output	10
7	Project Structure	10
8	Build and Usage	11
8.1	Requirements	11
8.2	Build	11
8.3	Usage	11
9	Conclusion	12

1. Abstract

CVM++ is a fully hand-written programming language runtime built from scratch in C++17 with zero external dependencies. It implements the complete pipeline from raw source text to executable output: a Lexer, a Recursive Descent Parser, an Abstract Syntax Tree, a Bytecode Compiler, and a Stack-Based Virtual Machine.

The project demystifies how high-level languages like Python, Java, and Lua are processed internally. Every component is written by hand — no parser generators, no LLVM, no runtime libraries — replicating the same architecture used by CPython and the JVM. The result is a working scripting language that supports integer and boolean data types, arithmetic, comparison operators, variables, if/else/else-if control flow, while loops, and built-in I/O, verified through seven comprehensive test scripts and compiled with zero warnings under `-Wall -Wextra`.

2. Problem Statement

Most developers use high-level languages like Python, JavaScript, or Java without understanding how raw source text is translated into instructions a CPU can execute. The layers between writing code and running it — lexical analysis, parsing, compilation, and virtual machine execution — are treated as black boxes.

This project addresses that gap by building every layer from scratch. The goal is not to produce a production-grade language, but to implement and deeply understand each stage of the compilation and execution pipeline using only C++ standard library primitives. This approach mirrors how foundational runtimes like CPython (Python's reference interpreter) and HotSpot (the JVM) were originally designed.

3. System Architecture

3.1 Pipeline Overview

The system is structured as a linear pipeline. Each stage transforms its input into a richer intermediate representation before passing it to the next stage.

Stage	Input	Output	File
Lexer	Raw source (.cvm)	Token stream	lexer.h/.cpp
Parser	Token stream	Abstract Syntax Tree	parser.h/.cpp
Compiler	AST	Bytecode + Name Table	compiler.h/.cpp
VM	Bytecode	Program output	vm.h/.cpp

3.2 Data Flow Diagram

```
Source Code (.cvm file)
|
```

```

v [ LEXER ] -- character-by-character scan
|
Tokens: [LET][IDENTIFIER:x][ASSIGN][NUMBER:10][SEMI]
|
v [ PARSER ] -- recursive descent
|
AST: LetStatement(x)
    L-- NumberLiteral(10)
    |
    v [ COMPILER ] -- AST tree-walk + backpatching
    |
Bytecode: 0: PUSH_INT 10
          5: STORE 'x'
          8: HALT
          |
          v [ VM ] -- fetch-decode-execute loop
          |
Output: variables = { x: 10 }

```

4. Component Design

4.1 Lexer — `lexer.h` / `lexer.cpp`

The Lexer performs lexical analysis. It scans the source string character-by-character and produces a flat `std::vector<Token>`. Each token carries its type (a `TokenType` enum), its raw text value, and a 1-based source line number for error messages.

Key design decisions:

- Single-pass scanning with a position pointer and `peek()` / `advance()` helpers.
- Whitespace and `//` line comments are silently skipped before each token.
- Keywords are identified by a string comparison table after reading a full identifier.
- Two-character tokens (`==`, `!=`, `<=`, `>=`) handled by peeking one character ahead.
- Line numbers tracked by incrementing a counter on every newline character consumed.

Category	Token Types
Literals	NUMBER, TRUE_LIT, FALSE_LIT
Identifiers	IDENTIFIER
Arithmetic	PLUS, MINUS, STAR, SLASH
Comparison	EQ_EQ, NOT_EQ, LESS, LESS_EQ, GREATER, GREATER_EQ
Assignment	ASSIGN
Delimiters	SEMICOLON, LPAREN, RPAREN, LBRACE, RBRACE
Keywords	LET, IF, ELSE, WHILE, PRINT, INPUT
Special	EOF_TOK

4.2 Parser — `parser.h` / `parser.cpp`

The Parser implements **Recursive Descent Parsing**. Each grammar rule maps to exactly one private method. This technique naturally encodes operator precedence through the call hierarchy: a lower-precedence rule always calls a higher-precedence rule before applying its own operators.

#	Method	Operators	Calls Into
1	<code>parseEquality()</code>	<code>== !=</code>	<code>parseComparison()</code>
2	<code>parseComparison()</code>	<code>< <= > >=</code>	<code>parseAdditive()</code>
3	<code>parseAdditive()</code>	<code>+ -</code>	<code>parseMultiplicative()</code>
4	<code>parseMultiplicative()</code>	<code>* /</code>	<code>parseUnary()</code>
5	<code>parseUnary()</code>	<code>- (unary)</code>	<code>parsePrimary()</code>
6	<code>parsePrimary()</code>	literals, <code>()</code>	—

A one-token lookahead (`IDENTIFIER` followed by `ASSIGN`) disambiguates variable assignment from expression statements without backtracking.

4.3 Abstract Syntax Tree — `ast.h`

The AST is a hierarchy of C++ structs all inheriting from a common `ASTNode` base class. `std::unique_ptr` is used throughout for automatic, exception-safe memory management — no manual `delete` calls exist anywhere in the codebase.

Node Type	Represents	Key Fields
<code>NumberLiteral</code>	Integer constant	<code>int</code> value
<code>BoolLiteral</code>	Boolean constant	<code>bool</code> value
<code>Identifier</code>	Variable reference	<code>string</code> name
<code>BinaryExpr</code>	<code>a OP b</code>	<code>string</code> op, left, right
<code>UnaryExpr</code>	<code>OP a</code>	<code>string</code> op, operand
<code>LetStatement</code>	<code>let x = e</code>	<code>string</code> name, initializer
<code>AssignStatement</code>	<code>x = e</code>	<code>string</code> name, value
<code>IfStatement</code>	<code>if/else-if/else</code>	condition, thenBranch, elseBranch
<code>WhileStatement</code>	while loop	condition, body
<code>PrintStatement</code>	<code>print e</code>	<code>expr</code>
<code>InputStatement</code>	<code>input v</code>	<code>string</code> varName
<code>Block</code>	<code>{ stmts }</code>	<code>vector<ASTNodePtr></code>

4.4 Bytecode Compiler — `compiler.h` / `compiler.cpp`

The Compiler performs a tree-walk over the AST and emits a flat sequence of bytecode instructions into a `std::vector<uint8_t>`. Variable names are stored in a separate name table (`vector<string>`) and referenced by 16-bit integer indices, keeping the instruction stream compact.

4.4.1 Jump Backpatching

Control flow (`if` / `while`) requires jump instructions whose target address is not known at emission time. Backpatching solves this in three steps:

1. Emit `JUMP` or `JUMP_IF_FALSE` with a 4-byte placeholder (`0x00DEAD00`).
2. Compile the body — now the target address equals the current code size.
3. Overwrite the placeholder in-place with the real target address.

This is the same technique used by production compilers including GCC and Clang.

4.4.2 Bytecode Encoding

All multi-byte operands are encoded in little-endian byte order (matching x86/x64):

- `PUSH_INT` operand: 4-byte `int32_t`
- `LOAD` / `STORE` / `INPUT` operand: 2-byte `uint16_t` name-table index
- `JUMP` / `JUMP_IF_FALSE` operand: 4-byte absolute bytecode offset

4.4.3 Compiled If Statement — Bytecode Layout

```
[ compile condition ]
JUMP_IF_FALSE  -> afterThen      (placeholder, patched later)
[ compile thenBranch ]
JUMP           -> afterElse      (only emitted when else exists)
afterThen:
[ compile elseBranch ]
afterElse:
```

4.4.4 Compiled While Statement — Bytecode Layout

```
loopStart:
[ compile condition ]
JUMP_IF_FALSE  -> afterLoop      (placeholder, patched later)
[ compile body ]
JUMP           -> loopStart      (absolute address, already known)
afterLoop:
```

4.5 Instruction Set Architecture (ISA)

Opcode	Hex	Operand	Stack Effect
PUSH_INT	0x01	4-byte int32	[] -> [int]
PUSH_BOOL	0x02	1-byte bool	[] -> [bool]
POP	0x03	—	[a] -> []
ADD	0x10	—	[a,b] -> [a+b]
SUB	0x11	—	[a,b] -> [a-b]
MUL	0x12	—	[a,b] -> [a*b]
DIV	0x13	—	[a,b] -> [a/b]
EQ	0x20	—	[a,b] -> [bool]
NEQ	0x21	—	[a,b] -> [bool]
LT	0x22	—	[a,b] -> [bool]
LEQ	0x23	—	[a,b] -> [bool]
GT	0x24	—	[a,b] -> [bool]
GEQ	0x25	—	[a,b] -> [bool]
NEGATE	0x30	—	[a] -> [-a]
LOAD	0x40	2-byte name idx	[] -> [value]
STORE	0x41	2-byte name idx	[value] -> []
JUMP	0x50	4-byte address	—
JUMP_IF_FALSE	0x51	4-byte address	[bool] -> []
PRINT	0x60	—	[value] -> []
INPUT	0x61	2-byte name idx	[] -> (stdin)
HALT	0xFF	—	stop

4.6 Virtual Machine — `vm.h` / `vm.cpp`

The VM implements a fetch-decode-execute loop over the compiled bytecode. Its execution model has three components:

- `ip` (instruction pointer) — index into the bytecode array.
- `stack` — `std::vector<Value>` used as a push-down operand stack.
- `variables` — `std::unordered_map<string, Value>` for the global scope.

4.6.1 Value Type

Runtime values use a plain C struct with a tagged union — `Type::INT` or `Type::BOOL`. A plain union is used instead of `std::variant` to avoid heap allocation and RTTI overhead in the hot execution loop.

4.6.2 Error Handling

The VM throws descriptive `std::runtime_error` for: stack underflow, undefined variable access, type mismatches on arithmetic operands, division by zero, and unknown opcodes.

All errors are caught at the pipeline entry point and printed with a [CVM ERROR] prefix and source line number.

5. Language Specification

5.1 Data Types

Type	Range	Example
Integer	32-bit signed	let x = 42;
Boolean	true / false	let flag = true;

5.2 Operators

Category	Operators	Notes
Arithmetic	+ - * /	Integer only. Division truncates toward zero.
Comparison	== != < <= > >=	Always returns a Boolean value.
Unary	-	Negates an integer.

5.3 Full Grammar

```

program      -> statement* EOF

statement    -> letStmt | assignStmt | ifStmt
              | whileStmt | printStmt | inputStmt | block

block        -> '{' statement* '}'
letStmt      -> 'let' IDENTIFIER '=' expr ';'
assignStmt   -> IDENTIFIER '=' expr ';'
ifStmt       -> 'if' '(' expr ')' block ( 'else' (ifStmt | block) )?
whileStmt    -> 'while' '(' expr ')' block
printStmt    -> 'print' expr ';'
inputStmt    -> 'input' IDENTIFIER ';'

expr         -> equality
equality     -> comparison ( ('==' | '!=') comparison )*
comparison   -> additive ( ('<' | '<=' | '>' | '>=') additive )*
additive     -> multiply ( ('+' | '-') multiply )*
multiply     -> unary ( ('*' | '/') unary )*
unary        -> '-' unary | primary
primary      -> NUMBER | 'true' | 'false' | IDENTIFIER | '(' expr ')'
```

5.4 Language Example

```

1 // Variable declaration
2 let x = 10;
3 let flag = true;
4
```

```

5 // Arithmetic and comparison
6 let result = (x + 5) * 2;
7
8 // If / else-if / else
9 if (result > 30) {
10     print 1;
11 } else if (result > 20) {
12     print 2;
13 } else {
14     print 3;
15 }
16
17 // While loop with accumulator
18 let sum = 0;
19 let i = 1;
20 while (i <= 10) {
21     sum = sum + i;
22     i = i + 1;
23 }
24 print sum;    // Output: 55
25
26 // Built-in I/O
27 input x;
28 print x * 2;

```

Listing 1: CVM++ language features

6. Test Results

All test scripts were compiled and executed with zero warnings under `g++ -std=c++17 -Wall -Wextra`. Results are fully deterministic.

Test Script	Features Tested	Key Output	Result
test1_arithmetic	All operators, precedence	2+3*4=14	PASS
test2_variables	let, reassignment, bool	Correct variable state	PASS
test3_conditionals	if/else-if/else, nested	Correct branch taken	PASS
test4_loops	while, nested, accumulator	sum=55, factorial=720	PASS
test5_fizzbuzz	All features combined	FizzBuzz 1–20 correct	PASS
test6_input	input, interactive I/O	Correct math on user input	PASS
fibonacci	while, multi-variable	0 1 1 2 3 5 8 13 21 34	PASS

6.1 Sample Program: Fibonacci Sequence

```

1 let a = 0;
2 let b = 1;
3 let i = 0;
4
5 while (i < 10) {
6     print a;
7     let temp = a + b;
8     a = b;
9     b = temp;
10    i = i + 1;
11 }

```

Listing 2: fibonacci.cvm

Output: 0 1 1 2 3 5 8 13 21 34

6.2 Debug Mode Output

Running any script with `-debug` exposes all four pipeline stages:

Listing 3: Debug mode for a simple let statement

```

$ ./cvm --debug tests/test2_variables.cvm

=== TOKENS ===
line 1  [let]  [x]  [=]  [10]  [;]

=== AST ===
Block
  LetStatement(x)
    NumberLiteral(10)

=== BYTECODE DISASSEMBLY ===
Name Table:  [0] = x
0: PUSH_INT  10
5: STORE     'x'
8: HALT

=== EXECUTION ===
[VM:0] PUSH_INT    stack: [] -> [10]
[VM:5] STORE 'x'   stack: [10] -> []
[VM:8] HALT

```

7. Project Structure

```

cvm/
|-- src/
|   |-- lexer.h / lexer.cpp      Tokenizer
|   |-- ast.h                   AST node definitions
|   |-- parser.h / parser.cpp    Recursive descent parser
|   |-- compiler.h / compiler.cpp Bytecode compiler + backpatching

```

```
| |-- vm.h / vm.cpp           Stack-based virtual machine
| |-- main.cpp               Entry point, REPL, debug tools
|-- tests/
| |-- test1_arithmetic.cvm
| |-- test2_variables.cvm
| |-- test3_conditionals.cvm
| |-- test4_loops.cvm
| |-- test5_fizzbuzz.cvm
| |-- test6_input.cvm
| |-- fibonacci.cvm
|-- CMakeLists.txt
|-- .gitignore
|-- README.md
```

File	Role	Key Classes / Functions
lexer.h/.cpp	Tokenizer	Lexer, Token, TokenType
ast.h	AST nodes	ASTNode and all node structs
parser.h/.cpp	Parser	Parser, parseStatement()
compiler.h/.cpp	Compiler	Compiler, Opcode, patchJump()
vm.h/.cpp	Virtual machine	VM, Value, run()
main.cpp	Entry point	main(), runREPL(), printAST()
CMakeLists.txt	Build config	CMake 3.10+, C++17

8. Build and Usage

8.1 Requirements

- g++ with C++17 support (GCC 7+ or Clang 5+)
- CMake 3.10+ (optional)
- Windows: MinGW-w64 / MSYS2 g++ 15.2 tested
- Linux / macOS: system g++ tested

8.2 Build

Listing 4: Build with g++ directly

```
g++ -std=c++17 -o cvm src/main.cpp src/lexer.cpp src/parser.cpp \
    src/compiler.cpp src/vm.cpp -Isrc
```

Listing 5: Build with CMake on Windows

```
mkdir build && cd build
cmake -G "MinGW Makefiles" ..
cmake --build .
```

8.3 Usage

Listing 6: Running CVM++

```
# Run a script
./cvm tests/test1_arithmetic.cvm

# Full debug output (tokens + AST + bytecode + execution trace)
./cvm --debug tests/test1_arithmetic.cvm

# Start interactive REPL
./cvm
```

9. Conclusion

CVM++ successfully implements a complete end-to-end programming language pipeline in C++17 with no external dependencies. Every stage — lexer, parser, compiler, and virtual machine — is hand-written from first principles, replicating the core architecture of real-world runtimes like CPython and the JVM.

All five project goals and all nine deliverables specified in the project brief are fully implemented and verified. The project goes beyond the minimum scope by adding additional comparison operators, else-if chains, unary negation, an interactive REPL, a bytecode disassembler, and line-numbered error messages.

Building CVM++ provided a ground-up understanding of how programming languages work internally — something no amount of using high-level languages can substitute for. Every design decision in the compiler and VM was driven by the same constraints that production language implementers face: how to represent programs efficiently in memory, how to encode control flow in a flat instruction stream, and how to execute that stream correctly without external frameworks.

— *End of Report* —

github.com/Nimit2504/CVM-plus-plus